

SimTIM: A Simulator for Temporal Cybersecurity Modeling

Anonymized authors

Anonymized affiliations

Abstract. Companies and other organizations are increasingly exposed to cyber threats. To effectively prepare their defense, organizations need to understand possible attacks and the potential implications of those attacks, as well as the effects of potential mitigation measures. These insights are necessary for making sound decisions on where to focus the organization's efforts: e.g., which nodes are especially critical and need hardening, how to optimize the network topology, how to most effectively implement intrusion protection etc.

This paper presents simTIM, a simulator that helps organizations obtain clarity about these aspects, thus enabling sound decision making on cyber defense. The simulator is model-based, built on the TIM meta-model introduced in recent work. simTIM allows the modeling of the organization's IT assets, potential attacks and their effects, and mitigations. simTIM supports probabilistic reasoning, temporal aspects, partial observability, and financial incentives. With simTIM, organizations can easily perform what-if analysis to assess the implications of different cyber defense strategies. The paper describes the design and implementation, and illustrates the practical use of simTIM.

Keywords: Cybersecurity · Cyberattack · Cyber defense · Simulation

1 Introduction (Zoltan)

Cybersecurity is a growing concern for companies and other organizations (e.g., public sector agencies, hospitals, universities etc.) [?]. Cyberattacks may cause significant damage to affected organizations, including financial losses, operational disruptions, and loss of reputation [?].

After decades of research and development in IT security, many techniques are known for making IT systems as secure as possible, for preventing, detecting, and mitigating cyberattacks [?]. However, implementing these security measures requires resources, including human experts, investments, and operational expenses. Organizations have limited resources. Thus, organizations need to consider carefully how to best use their resources to achieve the necessary level of protection [?]. For this, it is important to be able to answer questions such as:

- What damage could result from different types of cyberattacks?

- How beneficial would the implementation of different countermeasures be against those attacks?

Answering such questions is difficult. On the one hand, it requires combining different types of information, including information about the organization’s IT assets, about typical attacks, and about available countermeasures. On the other hand, it also requires sophisticated reasoning capabilities. Reasoning must cope with uncertainty (e.g., it is not known what attackers might do), with partial observability (e.g., some attacker actions may be detectable while others remain hidden), and temporal aspects (e.g., both attacks and defensive actions take time, and the success of an attack may depend on whether the attack or the defensive action finishes first).

Recently, a comprehensive meta-model for cybersecurity called TIM was introduced [?]. TIM defines the modeling constructs with which the necessary information on IT assets, attacks, and countermeasures can be captured. TIM also defines rules for reasoning with TIM models, supporting uncertainty, partial observability, and temporal aspects. As a proof of concept, TIM was also accompanied by a small program called simTIM, which showed the possibility of simulating TIM models [?].

However, the capabilities of simTIM are very limited. Not only does it implement only a subset of TIM, it offers no user interface, thus making it necessary to hard-wire all details of the model and of the simulated scenarios in the code. Further limitations of simTIM include a lack of support for non-trivial attack and defense strategies, a lack of support for model inspection, and a lack of support for visualizing simulation runs. For these reasons, simTIM is not helpful in assessing and improving the cybersecurity posture of an organization.

We solve this problem by introducing simTIM, a user-friendly simulator for cybersecurity based on TIM. simTIM makes the whole modeling and reasoning power of TIM accessible in an easy-to-use way. It features graphical modeling capabilities for modeling the organization’s IT assets and to inspect such models. It lets users easily capture information on vulnerabilities, possible attacks, and possible countermeasures. Simulations can be easily configured and run through the user interface, and the results of the simulations can be visualized in several ways, highlighting different aspects. Overall, simTIM is a versatile tool, with which the organization’s security team can easily analyze the organization’s current cybersecurity posture as well as the impact of potential cyber defense strategies, thus enabling sound decisions on improving cybersecurity.

2 Preliminaries (Zoltan)

This section gives a brief overview of the main features of TIM. For further details of TIM, including formal definitions and comprehensive examples, we refer the reader to [?].

Fig. ?? shows a compact overview of the core modeling concepts of TIM. In a TIM model of an organization’s IT assets, the *System* is composed of *Nodes*, which can represent, for example, servers or network devices like routers. A *Node*

Fig. 1. Overview of the static structure of TIM modeling constructs

has *Properties*, which can be used to capture further information about the node, such as the programs running on a server, or the type and amount of data stored in a database. A pair of *Nodes* may be connected through a *Link*. A TIM model contains, in the simplest case, two *Actors*: the attacker and the defender. (TIM also allows more fine-grained models, e.g., distinguishing between external and internal attackers.) An *Actor* can have different levels of *Access* to *Nodes* and *Links*: e.g., the defender may have administrator access to a server, while the attacker may have only user-level access or even no access to the same server.

Actions constitute the most important construct in TIM. Actions can represent both the individual steps performed by an attacker as part of an attack and the steps performed by the defender, implementing some countermeasure. An *Action* can be performed either on a *Node* or on a *Link*. An *Action* may lead to various changes, e.g., in the *Properties* of a *Node* or in the level of *Access* that the *Actor* has to a *Node* or *Link*. As an example, a successful privilege escalation attack on a *Node* may let the attacker gain administrator access to the given machine. Lateral movement of the attacker, on the other hand, can be modeled with link actions. *Actions* can also have preconditions, e.g., specifying that an attack *Action* is only possible if the target *Node* has some defined *Property*, like a specific version of an application that is known to be vulnerable.

An *Action* may be associated with a duration, a success probability, and a cost. The duration of *Actions* plays an important role in how TIM models may evolve over time: an *Actor* may start an *Action* at any time, provided that its precondition is fulfilled, and the effects of the *Action* realize after the given duration passed. The precondition must be fulfilled not only at the start of the *Action*, but throughout its duration. The success probability of an *Action* helps model the inherent uncertainty in cybersecurity: the *Action* takes effect with the given probability and is unsuccessful otherwise. The cost of *Actions* contributes to modeling economic aspects of cybersecurity. Successful attacks lead to a gain for the attacker and to a loss for the defender. These gains and losses, coupled with the costs for implementing *Actions*, lead to a financial balance for each *Actor*, which each *Actor* tries to optimize.

A TIM model may evolve over time. This is not only due to the effect of *Actions*, but also other events can happen. Basically, everything in Fig. ?? may change over time. For instance, new *Nodes* may join the *System* or existing *Nodes* may fail; *Properties* may change due to a software upgrade; new *Actions* may become available because new vulnerabilities have been discovered etc. TIM also foresees the possibility that the defender detects some ongoing *Actions* of the attacker, e.g., because an intrusion detection system raises an alarm.

3 Design

In this section we present the key considerations pertaining to the simulator’s design.

3.1 Usage

We developed simTIM to bridge the gap between theory and practice. It is a simulator based on the TIM model, written in Python, publicly available and open source under the Apache 2.0 License. Its intended users are security analysts, IT architects, and researchers who need to reason quantitatively about an organization’s cybersecurity posture. With simTIM, they can answer questions such as:

- How vulnerable is the current network against a given class of attacks?
- How much would a specific hardening measure or topology change improve the security posture?
- Which defense strategy yields the best trade-off between cost and risk reduction?

A typical workflow proceeds in five steps:

1. **Model the network** — create or import a graph of the organization’s IT assets with their properties.
2. **Configure actors** — choose the number of attackers and defenders, assign strategies, set capacity and budget.
3. **Select actions** — pick attack and defense action sets from the library or add custom ones.
4. **Run the simulation** — execute a configurable number of Monte Carlo runs, each for a specified duration.
5. **Analyze results** — inspect event histories, time-series plots, attack paths, and scenario comparisons.

The main design principle was extensibility. The whole project is structured in a way that makes changing and adding various simulation components easy and straightforward. For non-programmers, the GUI offers extensive customization: detailed network creation, actor configuration and more. Actions, networks and node properties are stored as JSON files. Those willing to dive deeper into the code can create attacker and defender strategies, actions, custom visualizations or their own detection engine. Supporting this purpose we provide base classes, validators and a modular architecture with clear extension points for each component.

3.2 Objectives

The simulation models actors on a network in a game-theoretical setup. There are *attackers* and *defenders*, each employing a *strategy* to meet their own objectives. Following the TIM model, the attacker’s objective is to maximize gain

minus cost, while the defender’s objective is to minimize damage plus cost. The simulator must support multiple concurrent attackers and defenders, each with independent strategies.

To align with realism, partial observability needs to be modeled. Attackers only see internet-exposed nodes initially and uncover more of the network through their actions. Defenders have full access to the network, but the detection of attacks is probabilistic.

The three quantitative dimensions the simulator captures are:

- **Security:** Every node has a per-actor access level (`NONE`, `VISIBLE`, `USER`, `ADMIN`). Links also carry per-actor access levels(`NONE`, `VISIBLE`).
- **Economics:** Every action has a cost. Successful attacks can cause *one-off* damage and gain upon completion, as well as the possibility to inflict *time-proportional* damage and gain that accumulates for as long as the attacker maintains access.
- **Time:** Actions have a duration. While an action executes, it occupies one of the actor’s limited capacity slots, its preconditions may be invalidated by concurrent actions, and it may be detected before completion.

Because every simulation run is stochastic a single run cannot provide reliable conclusions. The simulator is therefore designed around Monte Carlo repetition: users configure a number of independent runs with the same parameters, and the results are aggregated statistically. This enables meaningful comparison of strategies and network configurations despite the inherent randomness of individual runs.

3.3 Structure

The simulator can be structured into four main components: Core, GUI, Actors, and Network.

The *core* models the temporal behaviour of the simulation. A simulation consists of the continuous scheduling and executing of events; therefore, we need a data structure that ensures events are executed at the right time and in the right order. The core must also orchestrate all other components: loading actions, creating actors, initializing the network, running the economic model, and collecting results. An event-based communication mechanism is needed so that components remain decoupled. The detection engine, the economic model, and the history recorder should all react to state changes without being tightly coupled to the simulator loop.

The *GUI* should give users an intuitive entry point to configure and run simulations tailored to their needs. It should guide users through configuration, allow them to create and edit networks visually, and present meaningful analysis of the results.

The *actor* component must model capacity-limited, budget-constrained agents that make periodic decisions. Attackers maintain a set of visible and compromised nodes that expands as they progress; defenders have full network visibility

and receive probabilistic detection alerts. Both delegate their decision logic to a pluggable strategy, which the simulator must be able to discover and load without code changes.

The *network* component must represent a graph of nodes and links where each node carries typed properties (software versions, vulnerabilities, assets, services) that actions can check and modify. Crucially, actor access levels need to be tracked independent from one another, enabling scenarios with multiple simultaneous attackers operating at different stages of compromise on the same network.

In a typical simulation run these components interact as follows. The GUI collects the user's configuration and passes it to the core. The core's orchestration layer loads the chosen network, instantiates actors with their strategies, and loads and validates the selected actions. It then starts the simulator's event loop. Actors make decisions based on the selected strategy and submit the resulting events to the core. When an action completes, the core applies its postconditions to the network, publishes the state change on the event bus. This cycle continues until the simulation time elapses. After the simulation, the visualization components extract the data and process it.

3.4 Actions and Strategies

Actions are the primary mechanism through which actors change the state of the simulation. The simulator must represent actions as *data*, not code: each action is a declarative specification of its preconditions, postconditions, duration, cost, success probability, detection probability, and economic impact. This data-driven design is essential for extensibility. Users can add new actions by writing a JSON file rather than modifying source code.

Actions are divided into *node actions* and *link actions*. Node actions target a single node (e.g., privilege escalation, patching), while link actions target a connection between two nodes (e.g., lateral movement, firewall hardening). To ground the simulation in established security knowledge, actions are classified under the MITRE ATT&CK and D3FEND matrix. An example for an attack action is the "phishing" action. It maps to the MITRE ATT&CK technique T1566 and is classified under the initial access tactic (TA0001). The defense action "decoy file" maps to the D3-DF technique and the deceive tactic in the MITRE D3FEND matrix. The corresponding tactics are stored in the action JSONs and can be used by the strategy components.

Each actor follows a *strategy* that determines which action to perform, on which target, and when. Strategies must be pluggable: users should be able to add a new strategy by implementing a single class that conforms to a defined interface, without modifying any existing code. The simulator must discover available strategies automatically at startup. A strategy receives the actor's current view of the network and returns the next action to execute. This separation of decision logic from simulation mechanics allows independent development and comparison of strategies.

3.5 Detection

Each attack action carries a detection probability. Upon detecting an action, the defender is alerted and can react through its strategy. The timing of detection matters: an attack detected early can be countered before it completes, while a late detection may come after damage has been done.

The simulator must support different detection models, to accurately reflect real behaviour. The models determine *when* the action is detected, given it is detected. A Bernoulli trial determines whether the action is detected, based on the detection probability. If so, the detection time is given by a cumulative distribution function specific to the chosen model. The different detection models are based on different assumptions for example, whether detection becomes more likely the longer an action runs or if the detection is most likely at the start of the action.

Fig. 2. Diagram of the components

4 Implementation

4.1 Dependencies

simTIM is written in Python. The libraries numpy and matplotlib are the only production dependencies and are used for statistical computation and visualization. The GUI uses tkinter which ships with the python standard library.

4.2 Core

The heart of the core is the simulator engine. It is a priority queue implemented with heapq. It builds a min-heap sorted by time, so that the event with the smallest time will get executed next. In case of two events scheduled for the same time, the next event popped is decided via a predefined priority. Events always have a time, an event type and data. The time is used to order the events inside of the priority queue. The event type is used to classify the event. Event types include action success, action failed, property changed, etc. The data dictionary holds the actor, action, target node, and current access level. Above the simulation engine there is the orchestration layer. The simulation orchestrator loads network and actions, create actors, initializes the access and a simulator instance. Above that is a threading wrapper allowing sync or async execution and possibly multi-threading to reduce computation time, with a multitude of runs. The simulation produces its output through an event bus. Every state-changing operation publishes an observation event to the bus. A history recorder subscribes to all event types and captures them with timestamps. It is later used by the visualizer.

4.3 Action

Actions are stored as JSON files in a library folder. Categorizes into attack and defend, node and link actions. The JSON implements the TIM model by storing precondition, postcondition, cost, duration and success probability. Additionally, each file also defines detection probability, one-off and time-proportional damage and gain values and a classification under the MITRE ATT&CK framework for attacks or the D3FEND framework for defenses.

The preconditions and postconditions are expressed in a JSON-based condition language that supports compound logic, quantifiers, access level checks, software version comparisons, vulnerability presence checks, property checks and more. At load time, the action factory converts these specifications into callable Python functions.

Node actions evaluate their precondition and apply their postcondition on the same target node. Link actions evaluate their precondition against the link (or its start node) but apply their postcondition to the end node.

When a node's state changes (e.g., defender patches a vulnerability), the simulator re-evaluates preconditions of all ongoing actions targeting that node and aborts any whose preconditions became false. An action index is used to find actions in $O(k)$ via a lookup table.

4.4 Detection

Detection follows a two-step stochastic process. First, a Bernoulli trial with the action's detection probability ρ determines whether the action is detected at all. If so, the detection *time* is sampled from a cumulative distribution function $F_a(t)$ specific to the chosen detection engine, where $t \in [0, 1]$ is the normalized progress of the action. The sampled value is scaled by the action's duration to obtain the absolute detection time.

Three detection engines are provided:

- **Uniform:** $F_a(t) = t$. Detection is equally likely at any point during execution.
- **Early-weighted:** $F_a(t) = 1 - (1 - t)^n$. Biased toward early detection; most detections occur in the first fractions of the action's duration.
- **Late-weighted:** $F_a(t) = t^n$. Detection is biased toward the end of the action, modeling scenarios where evidence accumulates over time.

All engines extend a common base class that defines the interface: a CDF function, an inverse-CDF sampler, and the overall `calculate_detection_time` method. Adding a new engine requires only implementing these abstract methods. When an attack is detected, the event is published on the event bus. The defender alert observer forwards it to all defenders, who store it with a timestamp for use in their strategy.

4.5 Actor

As defined in the model *Attackers* only have limited access to the network. After creation every attacker gets a list of visible nodes by the orchestrator. Visible nodes are nodes that are exposed to the internet. *Defenders* get initialized with full admin access on every node and visible access on every link. After initialization every actor starts a decision run cycle. Decisions are made based on the strategy the actor is given. Actors are limited by capacity and budget. Capacity is the amount of actions the actor can run simultaneously. By default the capacity of attackers is infinite, while the capacity of the defender is 2. Budget is the amount of money, each actor is given at the start of the simulation.

4.6 Strategy

Three attacker strategies are provided:

- **Escalation:** Actions leading to greater access are prioritized. Once admin access is reached, the strategy shifts to link scanning actions, to discover adjacent nodes and the cycle repeats.
- **Greedy:** Every action is evaluated for highest expected gain, by multiplying gain with success probability and subtracting cost.
- **Random:** Randomly chooses an action from the list of available actions. Acts as a control to goal-oriented strategies.

Four defender strategies are provided:

- **Reactive:** Responsive actions falling under the eviction and isolation tactics are assigned the highest priority.
- **Proactive:** Actions classified under the harden tactic receive the highest priority scores.
- **Monitoring:** Emphasizes detecting and deceiving actions.
- **Balanced:** Combines strategies to use all tactics with only slightly biased priorities.

We acknowledge the limitations of the implemented strategies as developing suitable strategies was not a key concern of the development process.

Strategies are registered in a global strategy registry. This registry maps string names to strategy classes and is populated at startup with the built-in strategies. New strategies are added by implementing the abstract `get_priority` method on the appropriate base class and registering the class.

4.7 Network

The network is a graph consisting of nodes and links. Each node stores application names and versions, a list of known vulnerabilities, a list of assets, a properties and whether the node is exposed to the internet. Links connect two nodes and can be bidirectional or unidirectional.

Networks are stored as JSON files and processed through three layers: the **NetworkLoader** handles file I/O and path resolution (including a built-in library of networks), the **NetworkValidator** checks structural integrity (unique node IDs, valid link references, required properties) and the **NetworkFactory** constructs the **Network** object. The factory can also serialize a network back to JSON, enabling editing in the GUI's network creator.

4.8 Visualization

Visualization is built on multiple plot engines that share a common base class providing output management, axis formatting, and a colorblind-friendly theme.

The **TimeSeriesPlotEngine** produces plots from a single simulation run. It extracts three views from the event history: an *event timeline* showing attacker and defender actions as a scatter plot over simulation time, an *economic timeline* showing cumulative costs, gains, and damage as step-line charts, and a *node compromise timeline* showing the number of nodes at each access level as a stacked area chart.

The **ViolinPlotEngine** are used in the scenario comparison feature, where the user runs the same simulation under different configurations and compares the distributions. It produces violin plots for *damage distribution* across parameter values, showing how varying a simulation parameter affects outcomes.

The **AttackPathVisualizer** overlays per-run attack progression. For a selected simulation run, it replays the event history and colours each node according to the highest access level any attacker achieved on it. Links traversed by lateral movement actions are highlighted. A run selector lets the user step through individual runs.

All plots are rendered with matplotlib and can be embedded in the GUI's results window or saved to files.

5 Experience with using the simulator

5.1 Running a simulation

simTIM offers three modes of operation: a graphical user interface (default), an interactive command-line interface (`--cli`), and a quick demo mode (`--demo`) that runs a preconfigured simulation on the demo network and prints results to the terminal.

GUI workflow. The GUI is structured into tabs walking the user through every step of the configuration process. The *Simulation tab* is used to choose the number of runs and the duration of each run. In the *Network tab* the user can choose from a library of networks as well as load a network from a file on their computer. The chosen network can be visualized with a button that opens a new window. Another key feature of the Network tab is the *network creator*. Pressing the create network button opens a new window in which the user can create

their own network in the GUI without changing one line of code. In the network creator the user can create their own network from scratch or edit an existing one. When adding or editing nodes, a node window opens up, which prompts the user to specify the properties of the node. Editable properties are node id, name, category, assets, software, and exposure to the internet. There is also the option to create random or scale-free networks. The next two tabs are used to configure *actors*. The user can choose strategy, capacity and budget of a number of attackers and defenders. The *Action* tab lets the user choose a set of actions that can be used by attackers and defenders respectively. The *Scenario* tab can be used to compare a variable number of scenarios with a variable number of runs. After everything is set the *Overview* tab summarizes the simulation details and enables the user to start the simulation. A small window appears showing the progress of the simulation and after completing all runs, the results window is enabled.

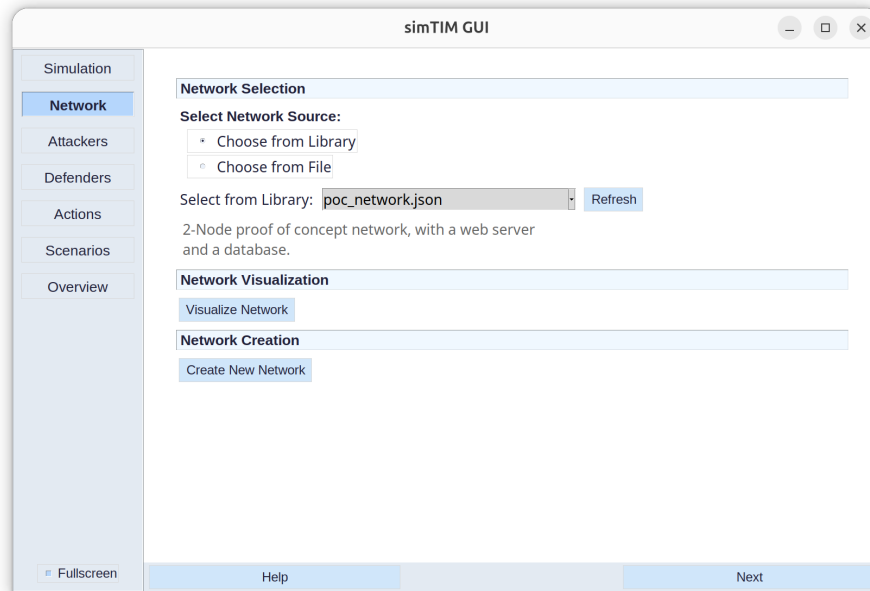


Fig. 3. Screenshot of the network tab in the simTIM GUI

Results. The *results window* shows a variety of visualizations of the simulation. The event history lists every event that occurred in chronological order for every run or filtered to each actor. The time series plots are a group of plots showing economic insights, node access level changes and events occurred depending on the simulation time. The attack path visualizer tab shows the path the attackers

took through the network, colouring nodes differently depending on the current access levels the attackers had on the pertaining node. If there have been scenario comparisons, a tab with violin plots shows the differences in economic impact based on the scenario.

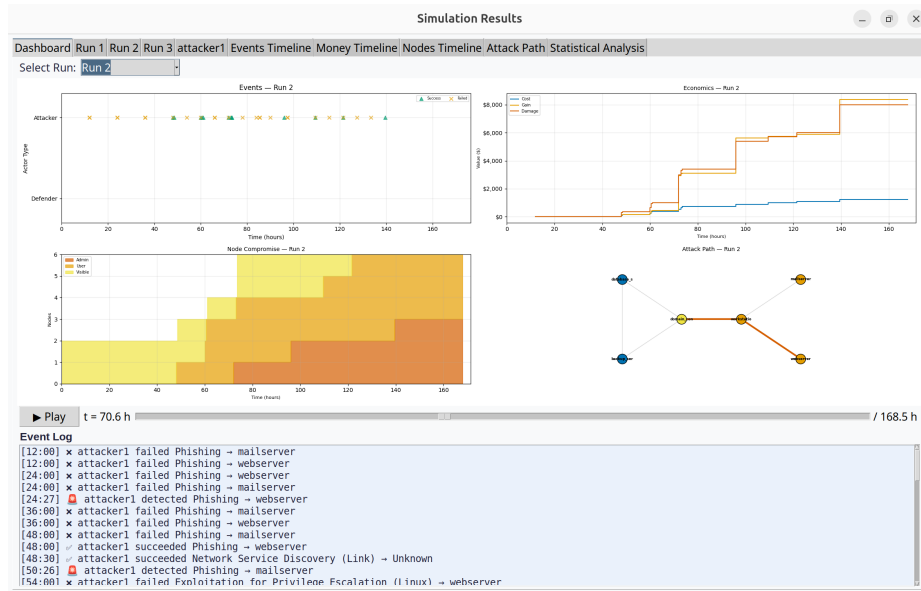


Fig. 4. Screenshot of different visualizations available.

5.2 Contributing to simTIM

simTIM is open-source software (Apache 2.0) designed with extensibility as a main design principle. The source code can be found in a public GitHub repository. [Link to GitHub] All four main extension points - actions, strategies, detection engines and networks - follow the same pattern: implement a small interface or provide a JSON file, place it in the correct directory, and the auto-discovery mechanism loads it in.

Adding networks. A new network is a JSON file placed in **networks/library/**. The format specifies a list of nodes (with properties such as software, vulnerabilities, assets, criticality, and exposure) and a list of links (with optional directionality). The network loader validates the topology, assigns default access levels, and makes the network available in the GUI's network selection dropdown. Using the GUI based Network Creator is a simpler alternative that yields the same results.

Adding actions. New attack or defense actions are defined as JSON files placed in `actions/library/attacks/` or `actions/library/defenses/`, respectively. Each file specifies a name, description, MITRE ATT&CK or D3FEND mapping, cost, duration, success probability, detection probability, damage and gain values, as well as preconditions and postconditions expressed in condition language. The action loader discovers all JSON files in these directories at startup, validates them, and registers them with the action registry.

Adding strategies. A new attacker or defender strategy is created by subclassing `AttackerStrategy` or `DefenderStrategy` from the strategy base module. The `get_priority()` method, which assigns a priority score to each candidate action-node pair, can be used to create strategies in line with the scoring system proposed by current implementations. The file is placed in `strategies/attackers/` or `strategies/defenders/`; the discovery module scans these directories and makes the strategy available in the GUI.

Adding detection engines. A new detection engine extends `BaseDetectionEngine` and implements three methods: `get_cdf_function()` (returning the cumulative distribution function $F_a(t)$ for detection timing), `sample_inverse_cdf()` (inverse-CDF sampling for Monte Carlo draws), and `get_configuration_summary()` (for logging). The file is placed in the `detection/` directory following the naming convention `*_detection.py` and is auto-discovered.

5.3 Experiments

6 Related work (Zoltan)

Kavak et al. categorized existing simulation efforts in cybersecurity according to purpose (representative environment, test and evaluation, training, risk analysis, humans in security), coverage (targets, threats, prevention), and type (model, platform) [?]. According to this taxonomy, simTIM covers all purposes and all coverage options, making it more comprehensive than any comparable work.

In recent years, several simulators have been proposed for security in specific domains, such as connected vehicles [?], industrial control systems [?], and power systems [?]. Compared to these, simTIM offers a much more versatile tool that can be applied to the security of any organization. Other research efforts focused on specific techniques, such as artificial intelligence [?], nature-inspired defense [?], and gamification [?]. In contrast, simTIM is agnostic to the specific techniques used by attackers and defenders; such techniques could be incorporated into simTIM as actions and strategies.

7 Conclusions (Zoltan)

This paper presented simTIM, a cybersecurity simulator based on the TIM meta-model. By supporting probabilistic reasoning based on physical time, simTIM

allows organizations to gain a good understanding of cybersecurity risks, their impact, and possible countermeasures. The source code of simTIM is publicly available, allowing organizations to extend and fine-tune it to model their specific characteristics. In future work, simTIM could be extended with more advanced attack and defense strategies, making it the backbone of organizations' cybersecurity analysis activities.